



Matrícula	Nome
-----------	------

Questão 1 – Responda falso, verdadeiro ou não sei. Duas erradas anulam uma certa. **Em caso da resposta ser falso, justifique porque ela é falsa.** Boa prova
[7,5]

1. A representação em árvore de uma expressão aritmética determina a ordem exata de avaliação da expressão.	F
2. A notação prefixada polonesa de Cambridge, utilizada em LISP requer conhecer a aridade dos operadores.	V
3. Em máquinas baseadas em pilha, o código correspondente a uma expressão aritmética é representado em forma pré-fixada.	F
4. A avaliação preguiçosa de expressões requer uma substancial simulação por software por isso não é utilizada em C.	F
5. São exemplos de sequenciamento explícito a ordem física da sequência dos comandos e a precedência das operações em expressões.	F
6. Na implementação de ponteiros com endereços relativos o valor do ponteiro é um deslocamento em relação ao endereço base de algum bloco de armazenagem no <i>heap</i> onde o objeto de dados é alocado.	NS
7. Para a execução da expressão $A := B + FN(C)$; três operações de referenciamento são necessárias.	F
8. Com uso da regra da cópia o ponto de retorno do subprograma chamado contém dois ponteiros (ip,ep), que representam os velhos valores de CIP e CEP.	V
9. O suporte para execução de funções recursivas é a pilha de registro de ativação.	NS
10. Em C, podemos declarar como sendo da classe de armazenamento <i>static</i> parâmetros de funções que são passados <i>by value</i> (por valor)	NS

1) Falso, pois a LP pode percorrer a árvore gerada de diferentes maneiras, e portanto pode gerar resultados diferentes.

3) A forma pós-fixada é utilizada em máquinas baseadas em pilha, e não a pré-fixada.

4) Embora realmente necessite de muita simulação por software, C suporta a avaliação preguiçosa - apenas não é o default, ou seja, é recomendado a passagem por valor.

Justificativas para as respostas respondidas como **falso**. Indique o número da pergunta e a seguir apresente a sua justificativa.

Responda apenas a uma questão dentre as abaixo. Não utilize exemplos apresentados em sala de aula.

Nota máxima 3,0

Estarei respondendo a questão 2.



Matrícula	Nome
-----------	------

Questão 2 – Descreva como operam os seguintes métodos para transmissão de parâmetros: *call by name*, *call by reference*, *call by value*, *call by value-result*, *call by constant value*, e *call by result*. (1,25). Apresente pequenos programas em C para ilustrar a transmissão de parâmetros para esses métodos (1,75)
[3,0]

Call by name: Parâmetros esperados são trocados pelas variáveis sendo passadas, sem necessitar avaliação gulosa ou passagem por valor. Por exemplo:

```
int funcao(int* a, int b) {  
    return *a + b;  
}  
  
int main() {  
    int i = 12;  
    printf("%d\n", funcao(&i, 10));  
    return 0;  
}
```

Neste exemplo, quando usamos `funcao(i, 10)`, é como se o parâmetro “a” dentro da função fosse substituído pela variável `i`, tornando o `return` na verdade um “`return i + 10;`”.

Call by reference: É passado para a função a referência de outra variável, desta forma não necessitando a criação de novas variáveis.

```
void funcao(int* a) {  
    *a = 10;  
}  
  
int main() {  
    int a = 0;  
    funcao(&a); //passa o endereço da variável a  
    printf("%d\n", a); //irá printar 10, pois a função acima modificou o valor de 0 para 10.  
    return 0;  
}
```

Neste exemplo, `funcao(int* a)` não cria uma variável local simplesmente para receber o valor passado, mas sim o endereço de uma variável já existente, para trabalhar com isto.



Matrícula	Nome
-----------	------

Call by value: a função recebe apenas o valor de uma outra variável ou constante já existente (recebe o r-value), e cria uma variável local (uma cópia) contendo este r-value.

```
void funcao(int a) {  
    print(“%d\n”, a);  
}  
  
int main() {  
    int valor = 0;  
    funcao(valor);  
}
```

Neste exemplo, ao chamar a função “funcao”, é passado o valor da variável “valor”, ou seja, o valor 0. Ao entrar na função, é então criada uma variável chamada a, que copia o valor passado (0) para o seu r-value. Seria semelhante a executar “int a = 0;” dentro da função.

Call by value-result: praticamente a mesma coisa do call by value, é criada uma variável local (cópia) em que o r-value é igual a variável original, porém ao final da execução este r-value é passado para a variável original (copia-se o r-value da variável local para a variável passada)

Call by constant value: similar ao call by value, porém o valor passado não pode ser alterado dentro da função. Utilizando o exemplo acima, seria como se substituíssemos “a” pelo valor passado, tendo portanto uma constante que não pode ser modificada em tempo de execução:

```
void funcao(const int a) {  
    print(“%d\n”, a);  
}  
  
int main() {  
    int valor = 0;  
    funcao(valor);  
}
```

Ou seja, foi passado o valor na variável “valor” da main, 0, como uma constante. Seria inválido executar “a = 13” dentro deste exemplo.

Call by result: o objetivo da call by result é que seja passada uma variável (no escopo chamador) que irá receber um novo valor após a execução da função. Um exemplo disso são as variáveis declaradas como “out” em ADA - estão ali apenas para que a função altere o r-value da variável passada.



Matrícula	Nome
-----------	------

Questão 3 – Considere uma LP com **checagem de tipo dinâmica**, **ambiente não local implícito** definido por **escopo dinâmico**. Conceitue regra da associação mais recente (0,5). Apresente um exemplo (0,5) para ilustrar a implementação de *pilha central* (0,5) e de *tabela central* (0,5) para referenciamento local e para não local. [2,0]

Questão 4. Considere a comunicação entre subprogramas por meio de **ambiente não local implícito**, **regra de escopo estático** em LP **estruturada em blocos** e **checagem de tipo estática**. Apresente um exemplo (0,5) para ilustrar a implementação *cadeia estática* (1,0) e *vetor mostrador* (1,0) para referenciamento local e não local. [2,5]